

Seguridad en Infraestructura

Pruebas de seguridad SAST

Comparación de Semgrep y Bandit

Licenciatura en Seguridad Informática
2026

Trabajo realizado a partir del contenido de la Clase 3.

1. Tipo de prueba elegido: SAST

Para este trabajo se eligió Static Application Security Testing (SAST). Según la Clase 3, SAST analiza el código fuente en busca de vulnerabilidades sin ejecutar el código. Esto permite realizar controles de seguridad durante el desarrollo y detectar determinados problemas antes de que la aplicación llegue a producción.

SAST se relaciona con el concepto de Shift Left, ya que busca incorporar la seguridad en etapas tempranas del ciclo de desarrollo. De esta manera, los problemas pueden detectarse mientras se está trabajando sobre el código, en lugar de descubrirlos recién cuando el sistema ya está desplegado.

El análisis estático no reemplaza al resto de las pruebas de seguridad. La misma clase diferencia SAST de DAST, IAST, SCA y pentesting. Cada una analiza aspectos diferentes de una aplicación.

2. Herramienta 1: Semgrep

Semgrep es una herramienta de análisis estático que permite buscar patrones potencialmente inseguros en el código. Una de sus principales características es que puede utilizarse con diferentes lenguajes, por lo que resulta útil en proyectos donde conviven varias tecnologías.

Característica	Semgrep Community Edition
Tipo de herramienta	SAST / análisis estático
Lenguajes	Más de 30 lenguajes. Entre ellos Python, Java, JavaScript/TypeScript, Go, C/C++, PHP, Ruby, Rust, Kotlin y C#.
Sistemas operativos	Linux, macOS y Windows para ejecución local.
Licencia	El motor open source utiliza LGPL-2.1.
Reglas	Permite utilizar reglas existentes y crear reglas propias.
CI/CD	Puede integrarse en pipelines para ejecutar el análisis automáticamente.
Principal ventaja	La variedad de lenguajes y la posibilidad de adaptar las reglas al proyecto.
Principal limitación	Los resultados dependen del lenguaje y de las reglas utilizadas; no detecta todos los tipos de vulnerabilidades.

Un ejemplo de instalación y ejecución básica es:

```
python3 -m pip install semgrep
semgrep --config=auto .
```

También puede utilizarse dentro de GitLab CI/CD para que el análisis se ejecute automáticamente cuando se realizan cambios en el repositorio.

3. Herramienta 2: Bandit

Bandit es otra herramienta SAST, pero está orientada específicamente a Python. Analiza el código construyendo un AST (Abstract Syntax Tree) y ejecuta diferentes plugins para buscar problemas de seguridad.

Característica	Bandit
Tipo de herramienta	SAST / análisis de seguridad para Python
Lenguaje	Python
Funcionamiento	Analiza el AST y utiliza plugins/tests para detectar problemas.
Instalación	Puede instalarse mediante pip y también utilizarse en contenedores.
Reportes	Puede generar resultados en distintos formatos, entre ellos JSON y SARIF.

Característica	Bandit
Configuración	Permite configurar el análisis y utilizar plugins.
CI/CD	Puede ejecutarse como parte de un pipeline.
Principal ventaja	Es sencilla y está especializada en seguridad de código Python.
Principal limitación	Está limitada al lenguaje Python.

Uso básico:

```
pip install bandit  
bandit -r .
```

4. Comparación entre las dos herramientas

Característica	Semgrep	Bandit
Alcance	Multilenguaje	Python
Tipo de análisis	Análisis estático basado en patrones y estructura del código	Análisis estático basado en AST y plugins
Código abierto	Sí	Sí
Personalización	Reglas propias	Configuración y plugins
Uso en CI/CD	Sí	Sí
Mejor escenario	Proyectos con varios lenguajes	Proyectos desarrollados en Python

5. Ejemplo de análisis

Un ejemplo sencillo de código Python que puede presentar un problema de seguridad es el siguiente:

```
import subprocess

comando = input("Ingrese comando: ")
subprocess.call(comando, shell=True)
```

El problema está en que una entrada proporcionada por el usuario termina utilizándose para ejecutar un comando del sistema. Una herramienta SAST puede detectar este tipo de construcción directamente desde el código, sin necesidad de ejecutar la aplicación.

El objetivo de este ejemplo es mostrar la idea básica de SAST: detectar patrones potencialmente inseguros durante el desarrollo.

6. Integración con CI/CD

Una ventaja de estas herramientas es que pueden incorporarse al pipeline de desarrollo. En lugar de ejecutar el análisis únicamente de forma manual, puede configurarse para que se ejecute automáticamente cada vez que se realiza un cambio.

Un flujo sencillo podría ser:

```
Código → pruebas → SAST → revisión → build/despliegue
```

En un proyecto Python se puede utilizar Bandit para realizar un análisis específico del lenguaje y Semgrep como herramienta complementaria. Si se encuentran problemas importantes, el pipeline puede configurarse para impedir que el cambio continúe hasta que sean revisados.

7. Ventajas y limitaciones de SAST

Ventajas	Limitaciones
Permite detectar problemas temprano.	Puede producir falsos positivos.
No necesita ejecutar la aplicación.	Puede dejar pasar problemas que no puede reconocer.
Puede automatizarse en CI/CD.	La cobertura depende de la herramienta y sus reglas.
Ayuda a aplicar el enfoque Shift Left.	No demuestra por sí solo que una aplicación sea segura.

8. Relación con otras pruebas de seguridad

SAST es solamente una parte de las pruebas de seguridad. En la Clase 3 también se presentan otras técnicas que tienen objetivos diferentes:

Tipo	Qué hace
SAST	Analiza el código sin ejecutar la aplicación.
DAST	Prueba la aplicación mientras está ejecutándose, simulando ataques externos.
IAST	Analiza la aplicación durante su ejecución utilizando instrumentación.
SCA	Analiza componentes y librerías open source y sus riesgos.
Pentesting	Simula ataques para buscar vulnerabilidades explotables.

9. Conclusión

SAST es una técnica útil para incorporar seguridad desde las primeras etapas del desarrollo. Su principal característica es que permite analizar el código sin ejecutar la aplicación, por lo que puede integrarse fácilmente al trabajo cotidiano de los desarrolladores y a los pipelines de CI/CD.

Entre las herramientas analizadas, Semgrep se destaca por su alcance multilenguaje y por la posibilidad de crear reglas propias. Bandit, en cambio, está específicamente orientada a Python y ofrece una alternativa sencilla para analizar este tipo de proyectos.

Para un proyecto Python, ambas herramientas pueden utilizarse de manera complementaria. De todas formas, ningún SAST detecta todos los problemas de seguridad, por lo que debe formar parte de un conjunto más amplio de controles de seguridad durante el ciclo de desarrollo.

10. Fuentes consultadas

- Material de Clase 3 — Seguridad en Infraestructura, UTN Facultad Regional Córdoba.
- Semgrep — documentación oficial y repositorio del proyecto: <https://semgrep.dev/> y <https://github.com/semgrep/semgrep>
- Bandit — documentación oficial: <https://bandit.readthedocs.io/>
- PyCQA Bandit — repositorio del proyecto: <https://github.com/PyCQA/bandit>